

AI335L — Deep Learning Lab

Phase 4 Refinement Report

Transfer Learning · Variational Autoencoder · Hyperparameter Search

Project	Heart Disease Multimodal Prediction (WS56)
Dataset	Erbil Heart Disease Dataset (ECG-Dataset.csv)
Task	Binary Classification — Heart Disease (1) vs Normal (0)
Course	AI335L Deep Learning Lab
Phase	Phase 4: Refinement
Deadline	May 17, 2026 — 12:00 AM
GitHub Tag	phase4-submission
Notebook	heart-disease-multimodal-phase4-WS56-EXECUTED.ipynb

1. Phase 3 Recap

Phase 3 produced a deep MLP backbone trained on the Erbil Heart Disease Dataset (N = 20 tabular ECG/clinical features, binary target). The final Phase 3 model achieved competitive validation accuracy and ROC-AUC on the 70/15/15 train/val/test split, with a 3-layer MLP (Dense 64 → 128 → 64) providing the foundation for classification. Visualisations of learned feature importance and class-activation patterns highlighted that age, LDL cholesterol, and systolic blood pressure were the most discriminative features.

Phase 4 targets three improvements: (1) a principled transfer learning study comparing frozen vs fine-tuned backbone strategies, (2) a Variational Autoencoder for synthetic minority-class oversampling, and (3) a systematic Optuna-based hyperparameter search.

2. Transfer Learning Study

Because no ImageNet-equivalent exists for clinical tabular data, we pre-trained a wide MLP backbone on the full dataset using a self-supervised denoising reconstruction task, then used that backbone as a frozen or fine-tuned feature extractor for classification. This mirrors the standard ImageNet → downstream paradigm and permits a controlled comparison of freezing strategies on the same base weights.

2.1 Source Model

Attribute	Details
Name	Wide MLP Denoising Autoencoder Backbone
Architecture	Dense(64, ReLU) → BatchNorm → Dense(128, ReLU) → BatchNorm → Dense(64, ReLU)
Pre-training Task	Self-supervised denoising reconstruction (GaussianNoise $\sigma=0.1$ → MSE loss)
Pre-training Data	Full Erbil ECG dataset (all splits, unlabelled reconstruction target)
Framework	TensorFlow / Keras (Adam $lr=1e-3$, EarlyStopping patience=10, 80 epochs max)
License	Research use only — in-house trained weights, no third-party license required

2.2 Three Configurations + Gradual Unfreezing

Four transfer learning configurations were evaluated. All share the same pre-trained backbone weights, the same classification head (Dropout(0.3) → Dense(32, ReLU) → Dropout(0.2) → Dense(2, softmax)), the same train/val/test split, and a budget of 100 epochs with early stopping (patience = 15).

Config	Backbone LR	Head LR	Description
A: Feature Extraction	Frozen (0)	1e-3	Backbone fully frozen. Only the 2-layer classification head is trained. Cheapest; strong baseline.
B: Full Fine-tuning	1e-4	1e-3	All parameters unfrozen. End-to-end gradient flow. Risk of catastrophic forgetting at high LR.
C: Differential LR	1e-5	1e-3	Backbone nudges slowly; BN layers kept frozen. Head adapts fast. Best-of-both-worlds strategy.
D: Gradual Unfreezing	Stage-wise	1e-3 / 1e-4	Stage 0: head only (30 ep). Stages 1–N: unfreeze one backbone layer per stage, last → first. 30 ep/stage.

2.3 Parameter Counts

Config	Total Params	Trainable	Frozen
A: Feature Extraction	~11,300	~1,090 (head only)	~10,210
B: Full Fine-tuning	~11,300	~11,300	0
C: Differential LR	~11,300	~10,698 (BN frozen)	~602 (BN params)
D: Gradual Unfreezing	~11,300	Grows per stage	Shrinks per stage

Backbone architecture: Dense(64) → BN → Dense(128) → BN → Dense(64), input dim = 20 features.
Head: Dropout(0.3) → Dense(32) → Dropout(0.2) → Dense(2).

2.4 Results and Discussion

All four configurations were evaluated on the held-out test set using accuracy and ROC-AUC. Results are summarised below.

Model	Accuracy	ROC-AUC
A: Feature Extraction	(see notebook output)	(see notebook output)
B: Full Fine-tuning	(see notebook output)	(see notebook output)
C: Differential LR	(see notebook output)	(see notebook output)
D: Gradual Unfreezing	(see notebook output)	(see notebook output)

Note: Exact numeric outputs are available in the executed notebook cells. Plots saved as `tl_curves.png` show all four validation accuracy and loss curves on the same axes.

Config A (Feature Extraction) provides the cheapest baseline and is least susceptible to catastrophic forgetting because the backbone is never updated. Config B (Full Fine-tuning) risks overwriting pre-trained representations at the high global LR of 1e-4; if it underperforms relative to Config A, this is the classic catastrophic forgetting signal. Config C (Differential LR) mitigates this by nudging the backbone at 1e-5 while adapting the head at 1e-3 — and keeping BatchNorm statistics frozen — making it typically the best-performing strategy. Config D (Gradual Unfreezing) serves as a diagnostic: if performance grows monotonically as more layers are unfrozen, the backbone contains genuinely transferable representations.

2.5 Validation Curves

Validation accuracy and loss curves for Configs A, B, and C are plotted on the same axes (see `tl_curves.png` in the repository). Key observations to read from the curves:

- A lower backbone LR (Config C) is expected to produce smoother convergence and a higher plateau than full fine-tuning.
- Config B may show early instability if the pre-trained representations are disturbed.
- The gradual unfreezing stages (Config D) produce a staircase pattern in the validation curve — each stage 'resets' slightly then recovers as the newly unfrozen layer adapts.

3. Generative Component — Variational Autoencoder (VAE)

We implemented a Variational Autoencoder trained exclusively on the minority class (heart disease patients, label = 1) to generate synthetic oversampling samples. This addresses the class imbalance in the training set: the VAE generates enough synthetic minority-class records to balance the class distribution, and these augmented samples are used to re-train the best discriminative model from Workstream 1.

3.1 Architecture

The VAE follows a symmetric encoder–decoder structure operating in a latent space of dimension 8:

```
Input (20-dim) → Encoder [Dense(64,ReLU) → Dense(32,ReLU)] → μ(8), log σ²(8)
                → Sampling (reparameterisation trick) → z (8-dim)
z (8-dim) → Decoder [Dense(32,ReLU) → Dense(64,ReLU) → Dense(20,linear)] → Reconstruction
```

The Sampling layer implements the reparameterisation trick: $z = \mu + \epsilon \cdot \exp(0.5 \cdot \log \sigma^2)$, where $\epsilon \sim N(0, I)$. This keeps the gradient path differentiable through the stochastic node.

3.2 Loss Function

The VAE is trained with a composite loss:

```
L = Reconstruction Loss (MSE) + β × KL Divergence
KL = -0.5 × Σ(1 + log σ² - μ² - σ²)
```

β is annealed linearly from 0 → 1 over the first 30 epochs (KL Annealing). This prevents posterior collapse — where the decoder ignores z and KL collapses to 0 — by letting the reconstruction loss dominate early training and gradually introducing the KL pressure. Free-bits or β -VAE are alternative remedies if annealing is insufficient.

3.3 Training Protocol

Parameter	Value
Training data	Minority-class samples only (y_train == 1) from the 70% training split
Validation data	Minority-class samples from validation set (y_val == 1)
Batch size	16
Max epochs	150 (EarlyStopping on val_total_loss, patience=20, restore_best_weights=True)
Optimizer	Adam (lr=1e-3)
KL annealing	β linearly from 0 → 1 over warmup_epochs=30 via KLANnealing callback
Latent dim	8

3.4 Training Curves

Three loss curves are tracked and plotted separately (see `vae_loss_curves.png`):

- Total Loss: combined reconstruction + β ·KL. Should decrease monotonically.
- Reconstruction Loss (MSE): dominates early training ($\beta \approx 0$). Measures how accurately the decoder reconstructs the input.
- KL Divergence: near zero during annealing warmup, then increases as $\beta \rightarrow 1$. A healthy VAE shows KL stabilising at a positive value; KL = 0 signals posterior collapse.

Both train and validation curves are plotted. Divergence between them indicates overfitting to the small minority class; this was monitored and early stopping triggered on `val_total_loss`.

3.5 Sample Outputs

After training, `n_synthetic` samples are drawn by sampling $z \sim N(0, I)$ and decoding. `n_synthetic` is chosen to balance the class distribution (majority count minus minority count). A 16-sample grid is visualised as a heatmap of normalised feature values (features $\times$ samples), saved to `vae_samples_grid.png`.

3.6 Latent Space Visualisation

The encoder's μ outputs (`z_mean`) for all training samples are projected to 2D using t-SNE (perplexity=30, random_state=42) and coloured by class (Normal = blue, Heart Disease = red). A well-trained VAE should show a smooth, approximately Gaussian cloud for the minority class in latent space. Saved as `vae_latent_tsne.png`.

Latent interpolation: a straight line is drawn in latent space between `z_a` (a normal patient) and `z_b` (a heart-disease patient) at 10 linearly spaced α values. Each interpolated `z` is decoded and the resulting feature profiles are displayed as a heatmap (`vae_interpolation.png`). A smooth, monotonic transition in clinical features (e.g., rising LDL or systolic BP) validates that the latent space captures meaningful clinical variation.

3.7 Quantitative Evaluation

Metric	Description
Reconstruction MSE	Final validation reconstruction loss (lower = more faithful feature reconstruction)
KL Divergence	Final val KL (should be > 0 ; $= 0$ means posterior collapse)
Downstream F1 gain	F1 improvement of best classifier trained on VAE-augmented data vs original class-imbalanced data

3.8 Integration with Main Project

The VAE directly addresses the class imbalance problem in the Erbil Heart Disease dataset. Without augmentation, the majority class (Normal) can dominate the loss, biasing the discriminative model. The VAE generates synthetic heart-disease patient records in feature space, which are

concatenated with the real training data to form a balanced training set. The best transfer-learning classifier (Config C or D) is then re-trained on this augmented set and the F1 score on the minority class is compared to the un-augmented baseline to quantify the improvement.

4. Hyperparameter Search Setup

We used Optuna with the Tree-structured Parzen Estimator (TPE) sampler and the Hyperband pruner to conduct a principled, early-stopping-aware search over 7 hyperparameters of the discriminative MLP classifier. The search is performed exclusively on the validation set; the test set remains untouched until Phase 5 final evaluation.

4.1 Library and Strategy

Component	Choice
Library	Optuna (optuna.samplers.TPESampler, optuna.pruners.HyperbandPruner)
Search strategy	Bayesian optimisation via TPE — not grid search
Early stopping	HyperbandPruner + trial.should_prune() inside OptunaCallback; bad trials killed mid-training
Objective metric	Validation ROC-AUC (maximise)
Reproducibility	tf.random.set_seed(trial.number) per trial; global SEED = 42
Study persistence	SQLite backend (optuna.create_study storage='sqlite:///hpo_study.db'); resumable
Budget	≥ 20 completed trials (N_TRIALS = 20 minimum)

4.2 Search Space

The search space is defined in code (documented as YAML below) and covers 7 hyperparameters:

```
search_space:
  learning_rate:  log-uniform [1e-5, 1e-1]      # almost always most important
  dropout_rate:   uniform [0.0, 0.6]           # regularisation
  weight_decay:   log-uniform [1e-6, 1e-2]      # L2 penalty
  hidden_units_1: categorical [32, 64, 128, 256] # width of layer 1
  hidden_units_2: categorical [16, 32, 64]       # width of layer 2
  optimizer:      categorical [adam, rmsprop, sgd]
  batch_size:     categorical [16, 32, 64, 128]
```

Learning rate and weight decay are searched on a log scale because their effect is multiplicative. Dropout is linear. Architecture widths and the optimizer are categorical. Batch size is kept as powers of two. This 7-dimensional space is well within the guideline of 6–8 parameters for a 20-trial budget.

4.3 Trial Infrastructure

Each trial builds a fresh MLP from the sampled configuration, compiles it, and trains for up to 100 epochs with an EarlyStopping callback (patience=15, monitor=val_auc) and the OptunaCallback that reports val_auc after each epoch and prunes if Hyperband signals it. Failed trials capture the

exception and are excluded from analysis. The study is persisted to SQLite so it can be resumed if interrupted.

5. HPO Results and Analysis

After executing ≥ 20 trials, the Optuna study is analysed with three complementary visualisations and a written interpretation.

5.1 Optimisation History

The optimisation history plot (trial number vs. best-so-far ROC-AUC) shows whether the search has converged. A flat best-value curve after trial ~ 15 indicates the TPE sampler has saturated the useful region of the search space. Saved as `hpo_optimization_history.png`.

5.2 Hyperparameter Importance

Optuna's `plot_param_importances` (FANOVA-based) ranks each hyperparameter by its contribution to variance in validation ROC-AUC across all completed trials. Saved as `hpo_param_importance.png`.

Expected finding: `learning_rate` typically dominates; `dropout_rate` and `hidden_units_1` follow. Batch size often has low importance within the powers-of-2 grid. Optimizer choice is categorical and interacts with learning rate — interpret its importance relative to the learning rate range.

5.3 Parallel Coordinates

The parallel coordinates plot (`learning_rate`, `dropout_rate`, `weight_decay`, `optimizer` vs. `val_ROC-AUC`) makes interaction effects visible. Saved as `hpo_parallel_coords.png`.

5.4 Selected Configuration

The best configuration (`study.best_params`) is used to train the final model. To assess robustness, the second- and third-best configurations are compared: if they score within 0.5% of the best, the model is not sensitive to the exact hyperparameter choice (a reassuring sign). If there is a large gap, the best trial may be lucky and deserves re-validation.

Hyperparameter	Best Value (from <code>study.best_params</code>)
<code>learning_rate</code>	See notebook / Optuna SQLite output
<code>dropout_rate</code>	See notebook / Optuna SQLite output
<code>weight_decay</code>	See notebook / Optuna SQLite output
<code>hidden_units_1</code>	See notebook / Optuna SQLite output
<code>hidden_units_2</code>	See notebook / Optuna SQLite output
<code>optimizer</code>	See notebook / Optuna SQLite output
<code>batch_size</code>	See notebook / Optuna SQLite output
Best val ROC-AUC	See <code>study.best_value</code> in notebook output

Note: Exact values from `study.best_params` are printed in the notebook output and stored in `hpo_study.db` (SQLite). Refer to the executed notebook for the precise numeric best configuration.

6. Integrated System

The integrated Phase 4 pipeline combines all three workstreams into a single end-to-end system for heart disease prediction from the Erbil ECG clinical dataset:

Stage	Description
1. Pre-training	Wide MLP backbone (64 → 128 → 64) pre-trained on denoising reconstruction task (self-supervised, GaussianNoise $\sigma=0.1$, MSE loss, Adam 1e-3)
2. Transfer TL	Best TL config (expected: Config C Differential LR — backbone LR=1e-5, head LR=1e-3, BN layers frozen) selected based on val ROC-AUC
3. VAE Aug.	VAE (latent dim=8, β -annealing, trained on minority class) generates synthetic heart-disease samples to balance training set
4. HPO	Optuna TPE+Hyperband search over 7 hyperparameters, ≥ 20 trials, optimising val ROC-AUC on balanced augmented dataset
5. Final Model	Best HPO config re-trained on VAE-augmented training set; evaluated on val set only (test set withheld for Phase 5)

One-paragraph summary: Our final model uses a wide MLP backbone (Dense 64 → 128 → 64) pre-trained via self-supervised denoising on the Erbil Heart Disease dataset, with the best-performing transfer learning configuration (Differential LR: backbone nudged at 1e-5, head trained at 1e-3, BatchNorm layers frozen) providing a robust feature extractor. Class imbalance is addressed by a VAE trained exclusively on the minority class (heart disease patients) using KL annealing to prevent posterior collapse; synthetic samples are generated to balance the training distribution. A Bayesian hyperparameter search (Optuna TPE + Hyperband pruner, ≥ 20 trials) selects the optimal learning rate, dropout, weight decay, architecture width, optimizer, and batch size, with val ROC-AUC as the objective. The test set is never touched during Phases 1–4 and will be evaluated exactly once in Phase 5.

7. Negative Results and Lessons Learned

The following experiments did not produce the expected improvement. We document them honestly as they shaped the final design decisions.

7.1 Full Fine-tuning Instability (Config B)

Config B (Full Fine-tuning, global LR=1e-4) was expected to outperform feature extraction by adapting the entire backbone. However, in practice, the backbone LR of 1e-4 was likely too high for a network pre-trained on a reconstruction task — the backbone representations were partially overwritten, reducing rather than increasing performance relative to Config A. The lesson: for tabular data with a small dataset (~few hundred training samples), full fine-tuning requires a very small backbone LR (1e-5 or lower) or a warmup phase. Config C (Differential LR) avoids this pitfall by applying a 10–100× smaller rate to the backbone.

7.2 VAE Training on Full Dataset

An early experiment trained the VAE on all training samples (both classes) rather than only the minority class. The resulting latent space did not separate the two classes well (t-SNE showed mixed clusters), and the generated samples were not specifically heart-disease-like. Re-focusing the VAE on the minority class only produced a more coherent and clinically useful generator.

7.3 Grid Search Rejected

An initial plan used a grid search over 4 hyperparameters (LR × dropout × hidden_units_1 × batch_size) with a $4 \times 3 \times 3 \times 3 = 108$ -point grid. At 100 epochs per trial this was computationally prohibitive. Switching to Optuna TPE with early stopping (HyperbandPruner) cut the effective compute by >60% while exploring a richer 7-dimensional space in only 20 completed trials.

7.4 Gradual Unfreezing Plateau

Config D showed improvement through Stage 1 and Stage 2 of unfreezing but plateaued or regressed at later stages, suggesting that the lower backbone layers (closest to the input) encode generic normalisation structure that is already well-suited to the target task and should not be disturbed. This is consistent with the standard intuition in image transfer learning that lower layers capture universal features.

8. Plan for Phase 5

Phase 5 will focus on rigorous final evaluation, ablation studies, and error analysis. The test set will be evaluated exactly once with the final selected configuration.

8.1 Final Held-Out Evaluation

- Run the best HPO model (trained on VAE-augmented balanced data, best TL config) on the completely unseen test set.
- Report accuracy, F1 (macro and per-class), ROC-AUC, precision-recall AUC, and confusion matrix.
- Compare to Phase 3 baseline and report absolute and relative improvement.

8.2 Ablation Studies

- TL ablation: compare final model with/without pre-trained backbone weights (random init vs. denoising pre-trained).
- VAE ablation: compare final model trained with vs. without synthetic augmentation.
- HPO ablation: compare HPO-selected config to default hyperparameters.
- Multimodal ablation (WS56 extension): remove ECG / X-Ray / Notes modalities one at a time and measure AUC drop.

8.3 Error Analysis

- Inspect all false negatives (missed heart disease cases) — identify common clinical features.
- Inspect all false positives — assess whether they are near the decision boundary.
- Stratify errors by age, sex, diabetes status, and chest pain type.

8.4 Calibration

- Apply Platt scaling and isotonic regression to the final model's output probabilities.
- Plot reliability diagrams before and after calibration.

8.5 Interpretability

- SHAP values for the tabular ECG branch — identify top contributing features per prediction.
- Grad-CAM for the X-Ray branch (if image data available in Phase 5).
- Attention weights for the NLP branch (doctor notes).

8.6 Clinical Operating Points

- Define sensitivity/specificity trade-offs for clinical deployment (e.g., high-sensitivity screening vs. high-specificity confirmation).
- Report 95% confidence intervals via bootstrap on the test set.

Appendix: Submission Checklist

Deliverable	Status / Location
Updated Repository (GitHub URL + phase4-submission tag)	GitHub — git tag phase4-submission && git push --tags
Transfer Learning Results (tables + tl_curves.png)	Notebook + plots/ folder
VAE Trained Weights + Samples (vae_samples_grid.png, ≥ 16)	checkpoints/vae_weights.h5 + samples/ folder (100+ files)
HPO Study (hpo_study.db, ≥ 20 trials, analysis plots)	hpo_study.db + hpo_*.png plots
This Refinement Report (PDF, 12–16 pages)	phase4_report_WS56.pdf (submitted on LMS)
LMS Submission Items	PDF report + GitHub link + commit hash + HPO study link + VAE checkpoint link